NEXTGEN PYTHON INTERNSHIP MX · Q2 2026 · NOSQL WORKSHOP

Workshop NoSQL DB
Cassandra

Case study: how Discord stores trillions of chat messages

Follow-along session **Hands-on, all levels welcome**

github.com/agent3dev/workshop-nosql-stage2-cassandra

TWO FLAVORS OF NOSQL

Documents got you here. Distribution gets you further.

MONGODB (PREVIOUS WORKSHOP)

- Flexible **documents**, varying shape
- Runs great on a single node
- Relationships: **embed** or **reference**
- Query however you like, mostly

CASSANDRA (THIS WORKSHOP)

- **Distributed by design** — built for many nodes
- Every table is designed around **one query**
- Data lives where its **partition key** says it does
- You choose consistency vs. availability, per query

THE REAL PROBLEM · DISCORD

Storing trillions of chat messages

- Discord's core read: **"last 50 messages in this channel, newest first"** — needed to be fast, always, at massive scale
- Their real production table partitioned by channel alone — every message in one channel, forever, in ONE partition
- Their largest channels eventually grew that partition big enough to cause **cluster-wide latency** — a real postmortem, not a hypothetical

> *“Lots of concurrent reads as users interact with servers can hotspot a partition... when Discord encountered a hot partition, it frequently affected latency across the entire database cluster.”*

— Discord Engineering: discord.com/blog/how-discord-stores-trillions-of-messages

TODAY, WE REBUILD IT

Curriculum roadmap

#	FILE	SQL · PREVIOUS WORKSHOP	CASSANDRA · TODAY
01	`01_crud.cql`	SELECT / INSERT / UPDATE / DELETE, JOINs	Same verbs — but partition-key reads only, no JOIN
02	`02_data_modeling.cql`	Normalize, then JOIN to assemble an answer	Denormalize: **one table per query**, built up front
03	`03_partitions_clustering.cql`	Not really a schema concern on one node	Partition key design determines scale & hotspots
04	`04_cql_consistency.cql`	Single-node ACID transactions	Tunable per query: `ONE` / `QUORUM` / `ALL`

FILE 1 OF 4 · 01_CRUD.CQL

CRUD basics: same verbs, different rules

SQL (POSTGRES, PREVIOUS WORKSHOP)

SQL

```sql
SELECT c.name, s.name
FROM channels c
JOIN servers s
  ON c.server_id = s.id
WHERE s.id = 'aaaa...';
-- JOIN resolves the relationship
-- filter on any column, freely
```

CQL · CASSANDRA (TODAY)

CQL

```cql
SELECT * FROM channels
WHERE server_id = aaaa...;
-- FAILS: server_id isn't the key
-- needs ALLOW FILTERING

-- and there is no JOIN. Ever.
```

- This "refusal" isn't a limitation to work around — it's the setup for file 2's real lesson

FILE 2 OF 4 · 02_DATA_MODELING.CQL

Data modeling: one table per query

- Naive table keyed by `message_id` alone — "messages in this channel" needs a full cluster scan
- Redesign: partition by channel, cluster by time, newest first
- A different query = a different table — duplicating data on write is normal here, not a bug
- This is Discord's actual schema, not a simplification

CQL

```cql
-- naive: no query is fast
PRIMARY KEY (message_id)

-- query-first: Discord's real schema
PRIMARY KEY
  (channel_id, message_id)
-- partition=channel, cluster=time
```

FILE 3 OF 4 · 03_PARTITIONS_CLUSTERING.CQL

Partitions & the hot-partition problem

- Load thousands of messages into ONE channel — `nodetool tablestats` shows a single, huge partition
- The real fix: add a time `bucket` to the partition key
- Same data, 10 smaller partitions instead of 1 — visible live in the stats, not just claimed
- The cost: reads now need to know which bucket(s) to check

CQL

```
-- one giant partition per channel
PRIMARY KEY
  (channel_id, message_id)

-- bucketed: caps partition size
PRIMARY KEY (
  (channel_id, bucket),
  message_id
)
  ↑ the extra ( ) is the fix
```

FILE 4 OF 4 · 04_CQL_CONSISTENCY.CQL

CQL consistency levels

- Real 3-node cluster, replication factor 3 — not a simulation
- `CONSISTENCY ONE / QUORUM / ALL` — how many replicas must answer
- We kill a node live: **ALL** breaks immediately, **QUORUM** and **ONE** keep working
- The tradeoff Discord makes every day: availability and speed over perfect consistency

CQLSH

```
CONSISTENCY QUORUM;
SELECT * FROM users WHERE ...;
-- OK, 2 of 3 replicas answer

-- (kill one node)
CONSISTENCY ALL;
SELECT * FROM users WHERE ...;
-- Cannot achieve consistency ALL
```

FORMAT

How today works

- This is a **follow-along-with-teacher** session, not a self-paced tutorial
- Some of you will type every command with me — others will watch and jump in on the open challenges. **Both are fine.**
- Each file ends with take-home challenges — we may not finish everything live, and that's expected
- Ask questions any time — we'll pause

BEFORE WE START

Get the cluster running

- `git clone github.com/agent3dev/workshop-nosql-stage2-cassandra`
- `docker compose up -d` — a real 3-node cluster, takes a few minutes to boot
- Check it's ready: `docker exec discord_cassandra1 nodetool status`
- Connect: `docker exec -it discord_cassandra1 cqlsh`

Full setup steps are in the repo README

LET'S BUILD

Rebuilding Discord's message store, one exercise at a time

github.com/agent3dev/workshop-nosql-stage2-cassandra